UNIVERSIDAD TECNOLÓGICA DE HONDURAS

Generación de APK con React Native, Expo y EAS Build

Guía técnica paso a paso: configuración de perfiles nativos, solución de errores críticos de Git/permisos y despliegue de actualizaciones OTA.

Asignatura: Programación Móvil II | Entorno: Visual Studio Code & Expo Application Services

Gemini Notebook

PASO 1 DE 8 • PREPARACIÓN DEL ENTORNO

Instala la herramienta oficial de compilación Expo Application Services

EAS CLI permite interactuar con los servidores de compilación de Expo desde cualquier terminal local.

¿Qué es EAS CLI?

- Línea de comandos especializada para compilar proyectos nativos en la nube sin requerir Android Studio localmente.
- El modificador global (-g) registra el comando 'eas' en las variables de entorno de tu sistema operativo.
- Prerrequisito: Tener instalado Node.js (versión LTS recomendada) y el gestor de paquetes npm.
- Compatible con Windows PowerShell, Command Prompt, macOS Terminal y Linux Bash.

>_ Terminal (PowerShell / Bash)

```
# Ejecutar en cualquier consola:

$ npm install -g eas-cli

# Verificar la versión instalada:
$ eas --version

# Salida esperada:
eas-cli/14.x.x windows-x64 node-v20.x
```

Gemini Notebook

PASO 2 DE 8 • AUTENTICACIÓN

Inicia sesión para enlazar tu terminal local con tu cuenta en Expo.dev

EAS requiere autorización para consumir los minutos de compilación y registrar los certificados.

Gestión de Cuenta

- Debes contar previamente con una cuenta activa en la plataforma web oficial https://expo.dev.
- La autenticación se realiza una sola vez; las credenciales se almacenan localmente de forma segura.
- Permite que la terminal acceda al panel de proyectos, certificados y colas de compilación en la nube.
- Comprobación de sesión: Puedes ejecutar en cualquier momento 'eas whoami' para verificar el usuario activo.

>_ Terminal interactiva de Expo

```
$ eas login

# Asistente de inicio de sesión:
? Email or username: alumno_uth
? Ingrese su clave de acceso

# Confirmación de enlace:
> Logged in as alumno_uth

# Verificación:
$ eas whoami
> alumno_uth
```

Gemini Notebook

PASO 3 DE 8 • VINCULACIÓN DEL PROYECTO

Genera el projectId y crea la estructura base de compilación

Conecta el directorio local de la aplicación con los servicios remotos mediante 'eas build:configure'.

>_ Comando de inicialización

```
# Ejecutar en la raíz del proyecto:
$ eas build:configure

# Interacción con el asistente:
? EAS project not configured.
  Do you want to create a project on Expo? (Y/n) > Y

? Which platforms would you like to configure?
  > Android

✓ Created project on Expo
✓ Successfully configured eas.json
```

Modificaciones Automáticas

- **app.json**
  Se añade la clave 'extra.eas.projectId' con un identificador UUID único vinculado a tu cuenta en Expo.dev.
- **eas.json**
  Se crea el archivo de control maestro donde se definen los perfiles de compilación (development, preview, production).
- **Asignación de Plataforma**
  Al seleccionar Android, el asistente genera la plantilla específica para el sistema operativo móvil.

Gemini Notebook

RESOLUCIÓN DE PROBLEMAS • ERRORES EN PASO 3

Soluciona conflictos de inicialización, identidad y permisos de Git

EAS depende del control de versiones; aprende a diagnosticar y corregir los 3 bloqueos más frecuentes.

[!] CASO 1: Repositorio Requerido

```
Mensaje detectado:
'EAS requires you to use a Git
repository'
```

Causa: La carpeta del proyecto no tiene control de versiones inicializado.

SOLUCIÓN RECOMENDADA: Responder 'Yes' en el prompt de EAS o ejecutar directamente en la terminal:

$ git init

[!] CASO 2: Identidad no Configurada

```
Mensaje detectado:
'fatal: Please tell me who you
are'
```

Causa: Git requiere nombre y correo local o global para registrar el autor.

SOLUCIÓN RECOMENDADA: Configurar las credenciales en tu máquina:

$ git config --global user.name "Estudiante UTH"
$ git config --global user.email "alumno@uth.hn"

[!] CASO 3: Error 128 (Dubious Ownership)

```
Mensaje detectado:
'fatal: detected dubious
ownership in repository'
```

Causa: Conflicto de permisos en Windows cuando la carpeta se crea como Administrador y se ejecuta como usuario estándar.

SOLUCIÓN RECOMENDADA: Declarar la carpeta como segura en Git y reintentar:

$ git config --global --add safe.directory "C:/Ruta/Proyecto"
$ eas build:configure

Gemini Notebook

PASO 4 DE 8 • ARCHIVO EAS.JSON

Configura el perfil Preview para compilar un paquete APK instalable

Por defecto Expo compila en formato AAB para Google Play; debemos especificar el tipo APK.

>_ Archivo eas.json (Raíz del proyecto)

```
{
  "cli": {
    "version": ">= 20.4.0",
    "appVersionSource": "remote"
  },
  "build": {
    "preview": {
      "distribution": "internal",
      "android": {
        "buildType": "apk"   // <-- Línea obligatoria
      }
    },
    "production": {}
  }
}
```

Claves de la Configuración

- **Perfil 'preview'**

Diseñado para pruebas internas entre el equipo docente y los estudiantes sin publicar en tiendas oficiales.

- **Propiedad 'buildType'**

Indica al motor de compilación Gradle que empaquete un instalador binario autónomo (.apk).

- **Sintaxis Estricta**

Respetar la notación CamelCase ('buildType'). En formato JSON las comas y llaves son obligatorias.

- **Distribución Interna**

'distribution: internal' permite descargar e instalar la app directamente desde un enlace público.

Gemini Notebook

PASO 5 DE 8 • IDENTIDAD DE LA APLICACIÓN

Define el Application ID único en formato de dominio inverso

Android exige que cada paquete instalado en el sistema operativo posea un identificador único en el mundo.

Reglas del Package Name

- Formato de dominio inverso: 'com.institucion.nombreapp' o 'com.docente.proyecto'.
- Debe escribirse estrictamente en letras minúsculas, sin espacios ni caracteres acentuados.
- Se compone de al menos dos segmentos separados por un punto (ej: com.uth.appmovil).
- Una vez fijado en app.json, identifica la app en el dispositivo y evita sobreescritura accidental de otras apps.

>_ Terminal & Archivo app.json

```
# Pregunta del asistente EAS:
? What would you like your Android application id
to be?
  > com.docente.miproyecto

# Expo lo almacena automáticamente en app.json:
{
  "expo": {
    "name": "MiProyecto",
    "slug": "mi-proyecto",
    "android": {
      "package": "com.docente.miproyecto"
    }
  }
}
```

Gemini Notebook

PASO 6 DE 8 • COMPILACIÓN Y FIRMA DIGITAL

Lanza la compilación remota y genera el Android Keystore

EAS Build sube el código, crea la llave de firma digital en sus servidores y compila el instalador nativo.

>_ Comando de compilación para Android

```
$ eas build --profile preview --platform android

# Asistente de firma digital:
? Generate a new Android Keystore? (Y/n) > Y

# Proceso en progreso:
✓ Created Keystore and stored on Expo cloud
✓ Project archive compressed and uploaded
✓ Build queued on Expo cloud builders

# Enlace de seguimiento en la nube:
Build details: https://expo.dev
```

Firma Digital (Keystore)

- Android exige que todo paquete esté firmado criptográficamente para permitir su instalación en dispositivos reales.
- Al responder 'Yes', Expo genera automáticamente una clave segura y la almacena en sus servidores protegidos.
- No requieres gestionar archivos de certificados locales ni contraseñas manuales.
- La compilación se ejecuta en máquinas virtuales remotas de Expo, liberando la memoria RAM y el procesador de tu computadora.

Gemini Notebook

RESOLUCIÓN DE PROBLEMAS • ERROR EN SUBIDA

Supera el fallo de empaquetado y clonación temporal en Git (Error 128)

EAS falla al crear la copia 'shallow clone' si existen cambios pendientes o bloqueos de control de versiones.

Opción A: Confirmar Cambios en Git (Recomendada)

Guarda el estado actual del proyecto en el árbol de trabajo local para que el comando git clone de EAS procese los archivos limpiamente.

```
# 1. Agregar todos los archivos:
$ git add .

# 2. Confirmar commit:
$ git commit -m "Configuracion inicial EAS"

# 3. Reintentar compilación:
$ eas build --profile preview --platform android
```

Opción B: Omitir Git con EAS_NO_VCS

Fuerza a EAS CLI a comprimir la carpeta completa directamente en un archivo tarball sin validar ramas ni historiales de Git.

```
# En Windows PowerShell:
$env:EAS_NO_VCS=1
eas build --profile preview --platform android

# En Command Prompt (CMD):
set EAS_NO_VCS=1 && eas build --profile preview --platform android

# En Bash / Linux:
EAS_NO_VCS=1 eas build --profile preview --platform android
```

Gemini Notebook

PASO 7 DE 8 • DISTRIBUCIÓN

Descarga el instalador mediante enlace directo o código QR

Al concluir la compilación en la nube, Expo publica el instalador en su infraestructura de descarga.

Canales de Entrega

- **Código QR en Terminal**
  Al finalizar la compilación, se imprime un código QR directamente en consola para escanear con la cámara del celular.
- **Panel Web de Expo.dev**
  La sección 'Builds' almacena el historial de paquetes generados con tamaño, logs de compilación y botón de descarga.
- **Enlace Público Compartible**
  Genera un enlace HTTPS directo que puedes compartir con alumnos o evaluadores para descargar la aplicación sin intermediarios.
- **Instalación en Android**
  Al abrir el archivo, habilitar 'Instalar desde fuentes desconocidas' en la configuración del teléfono.

```
>_ Consola de Finalización

✓ Build finished successfully!

📦 Archivo de aplicación compilado
📱 Plataforma: Android

🔗 Enlace directo de descarga:
Disponible en el panel oficial de Expo.dev

Escanea el código QR generado en la terminal
desde tu dispositivo móvil Android para instalar:

 [ CÓDIGO QR GENERADO EN CONSOLA ]
 [ Escanea con la cámara del celular ]
 [ Descarga e instalación directa ]
```

Gemini Notebook

PASO 8 DE 8 • DESPLIEGUE CONTINUO

Publica cambios de código al instante con EAS Update

Envía mejoras de interfaz o correcciones de JavaScript sin obligar a reinstalar un nuevo paquete.

¿Por qué usar EAS Update?

- **Sin recompilación nativa**
  Las compilaciones nativas tardan entre 5 y 15 minutos. Un EAS Update se propaga en cuestión de segundos.
- **Actualización transparente**
  Cuando el usuario vuelve a abrir la app instalada en su teléfono, descarga el nuevo código de forma automática.
- **Control por ramas (branch)**
  El parámetro '--branch preview' asegura que la actualización impacte únicamente a los dispositivos vinculados al canal de pruebas.
- **Historial y mensajes**
  El parámetro '--message' documenta qué cambios incluye cada versión (ej: 'Actualización de interfaz').

>_ Comando de Actualización Rápida

```
# Ejecutar en la terminal del proyecto:

eas update --branch preview --message
"Actualización de interfaz"

# Salida de EAS CLI:
✓ Exporting project files...
✓ Uploading assets (images, fonts)...
✓ Publishing update group to branch...

[OK] Update published successfully!

# Los teléfonos con la app recibirán los
# cambios en el próximo reinicio.
```

Gemini Notebook